BAB III

Studi Kasus 1: Sorting

Algoritme sorting adalah metode yang digunakan untuk mengurutkan elemen-elemen dalam suatu himpunan data. Terdapat berbagai macam algoritme sorting yang dapat digunakan, masing-masing dengan kelebihan dan kekurangannya sendiri.

Algoritme *Bubble Sort* bekerja dengan membandingkan sepasang elemen yang bersebelahan dalam himpunan data dan menukar posisi jika diperlukan. Proses ini diulang secara berulang hingga semua elemen terurut. Pada setiap iterasi, elemen terbesar akan "muncul" ke bagian paling akhir. Proses ini terus berlanjut hingga seluruh himpunan data terurut.

Algoritme *Quick Sort* akan memilih elemen yang disebut sebagai "pivot" dari himpunan data. Setelah itu, algoritme membagi himpunan data menjadi dua bagian berdasarkan nilai *pivot*. Elemen-elemen yang lebih kecil daripada *pivot* ditempatkan di satu sisi, sedangkan elemen-elemen yang lebih besar ditempatkan di sisi lain. Proses ini berulang untuk kedua bagian himpunan data hingga seluruh elemen terurut.

Selain *Bubble Sort* dan *Quick Sort*, terdapat juga algoritme sorting lainnya seperti *Insertion Sort*, *Selection Sort*, dan *Merge Sort*. Setiap algoritme memiliki cara kerja yang berbeda namun tujuan akhirnya tetap sama, yaitu mengurutkan himpunan data dengan efisien.

```
for i:= 1 to N-1 do
  min = i
  for j:= i+1 to N do
    if A[j]< A[min] then
      min = j
    end if
  end for
swap(A[i],A[min])
end for
```

Gambar 3.1: Algoritme Selection Sort

3.1 Selection Sort

Algoritme *Selection Sort* adalah salah satu metode pengurutan sederhana untuk mengurutkan elemen-elemen dalam sebuah array. Ide dasarnya adalah memilih elemen terkecil (atau terbesar, tergantung pada kebutuhan) dari array dan menukar-

nya dengan elemen pertama. Kemudian, pilih elemen terkecil kedua dan tukar dengan elemen kedua, dan seterusnya. Proses ini terus berlanjut hingga seluruh array terurut.

Berikut ini adalah langkah-langkah dalam algoritme selection sort:

1. Mulai dengan array yang belum terurut.
2. Temukan elemen terkecil dalam array dan simpan posisinya.
3. Tukar elemen terkecil dengan elemen pertama.
4. Pindah ke elemen kedua dan temukan elemen terkecil dalam sisa array tersebut.
5. Tukar elemen terkecil dengan elemen kedua.
6. Lanjutkan proses ini hingga seluruh array terurut.

Dalam implementasinya, ada beberapa hal penting yang perlu diperhatikan:

1. Perulangan *nested* digunakan untuk membandingkan setiap elemen dengan elemen lainnya dalam array.
2. Penggunaan variabel sementara untuk menyimpan nilai terkecil dan posisinya.
3. Setelah menemukan elemen terkecil, tukar dengan elemen pertama menggunakan operasi pertukaran.

```
for i:= 1 to N-1 do
  min = i
  for j:= i+1 to N do
    if A[j]< A[min] then
      min = j
    end if
  end for
swap(A[i],A[min])
end for
```

Gambar 3.2: Algoritme Selection Sort

Meskipun algoritme *Selection Sort* cukup sederhana, namun kompleksitas waktunya adalah $O(n^2)$, di mana n adalah jumlah elemen dalam array. Artinya, semakin banyak elemen dalam array, semakin lama waktu yang dibutuhkan untuk mengurutkannya.

Gambar 3.3 menunjukkan contoh implementasi dari algoritme selection sort. Penjelasan detil terkait tahap-1 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

Kondisi Awal

1	2	3	4	5	6	7
16	22	25	3	9	14	23

SELECTION SORT

Tahap-1

	2	3	4	5	6	7
	22	25	16	9	14	23

Tahap-2

1	2	3	4	5	6	7
3	9	25	16	22	14	23

Tahap-3

1	2	3	4	5	6	7
3	9	14	16	22	25	23

Tahap-4

1	2	3	4	5	6	7
3	9	14	16	22	25	23

Tahap-5

1	2	3	4	5	6	7
3	9	14	16	22	25	23

Tahap-6

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Hasil Akhir

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Gambar 3.3: Ilustrasi algoritme Selection Sort

1. Nilai i=1. Nilai variabel min=1.
2. Nilai A[min]=A[1] dibandingkan dengan A[2]=22, A[min]<A[2].
3. Nilai A[min]=A[1] dibandingkan dengan A[3]=25, A[min]<A[3].
4. Nilai A[min]=A[1] dibandingkan dengan A[4]=3, A[1] > A[4]. Maka min=4.
5. Nilai A[min]=A[4] dibandingkan dengan A[5]=9, A[min] < A[5].
6. Nilai A[min]=A[4] dibandingkan dengan A[6]=14, A[min]<A[6].
7. Nilai A[min]=A[4] dibandingkan dengan A[7]=23, A[min]<A[7].
8. Tukar A[i]=A[1] dengan A[min]=A[4].

Penjelasan detil terkait tahap-2 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

1. Nilai i=2. Nilai variabel min=2.
2. Nilai A[min]=A[2]=22 dibandingkan dengan A[3]=25, A[min]<A[3].
3. Nilai A[min]=A[2]=22 dibandingkan dengan A[4]=16, A[min]>A[4]. Maka min = 4.
4. Nilai A[min]=A[4]=16 dibandingkan dengan A[5]=9, A[min] > A[5]. Maka min=5.
5. Nilai A[min]=A[5]=9 dibandingkan dengan A[6]=14, A[min] < A[6].
6. Nilai A[min]=A[5]=9 dibandingkan dengan A[7]=23, A[min]<A[7].
7. Tukar A[i]=A[2] dengan A[min]=A[5].

Penjelasan detil terkait tahap-3 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

1. Nilai i=3. Nilai variabel min=3.
2. Nilai A[min]=A[3]=25 dibandingkan dengan A[4]=16, A[min]>A[4]. Maka min = 4.
3. Nilai A[min]=A[4]=16 dibandingkan dengan A[5]=22, A[min]<A[5].

4. Nilai A[min]=A[4]=16 dibandingkan dengan A[6]= 14, A[min] > A[5]. Maka min=6.
5. Nilai A[min]=A[6]=14 dibandingkan dengan A[7]=23, A[min] < A[7].
6. Tukar A[i]=A[2] dengan A[min]=A[5].

Penjelasan detil terkait tahap-4 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

1. Nilai i=4. Nilai variabel min=4.
2. Nilai A[min]=A[4]=16 dibandingkan dengan A[5]=22, A[min] < A[5].
3. Nilai A[min]=A[4]=16 dibandingkan dengan A[6]=25, A[min] < A[6].
4. Nilai A[min]=A[4]=16 dibandingkan dengan A[7]=23, A[min] < A[7].
5. Tukar A[i]=A[4] dengan A[min]=A[4].
6. Berarti tidak ada perubahan.

Penjelasan detil terkait tahap-5 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

1. Nilai i=5. Nilai variabel min=5.
2. Nilai A[min]=A[5]=22 dibandingkan dengan A[6]=25, A[min] < A[6].
3. Nilai A[min]=A[5]=22 dibandingkan dengan A[7]=23, A[min] < A[7].
4. Tukar A[i]=A[5] dengan A[min]=A[5].
5. Berarti tidak ada perubahan.

Penjelasan detil terkait tahap-6 dari algoritme selection sort berdasarkan ilustrasi pada Gambar 3.3 adalah sebagai berikut:

1. Nilai i=6. Nilai variabel min=6.
2. Nilai A[min]=A[6]=25 dibandingkan dengan A[7]=23, A[min] > A[7]. Maka min = 7.
3. Tukar A[i]=A[6] dengan A[min]=A[7].

3.2 Insertion Sort

Algoritme *Insertion Sort* adalah salah satu metode pengurutan yang sederhana namun efektif. Cara kerja algoritma *Insertion Sort* adalah menyisipkan satu per satu bilangan dalam sebuah deretan ke posisi yang benar. Mula-mula algoritma melakukan perulangan i dari nilai 2 hingga N. Dalam perulangan ke-1 algoritma akan membandingkan bilangan yang ke-i dengan semua bilangan di sebelah kirinya, kemudian menyisipkannya *(insert)* ke lokasi yang benar.

```
for I = 2 to N do
  iDataSisip = A[i]
  j = i
  while (j > 1) and (A[j-1] > iDataSisip) do
     A[j] = A[j-1]
     j = j-1
 end while
 A[j] = iDataSisip
end for
```

Gambar 3.4: Algoritme Insertion Sort

Algoritme *Insertion Sort* memiliki kompleksitas waktu $O(n^2)$, di mana n adalah jumlah elemen dalam daftar. Meskipun demikian, *Insertion Sort* efisien untuk daftar yang sudah hampir terurut atau memiliki urutan yang kecil. Algoritme ini juga cukup sederhana untuk diimplementasikan dan bekerja dengan baik untuk daftar dengan jumlah elemen yang relatif kecil.

Ketika menggunakan *Insertion Sort*, kita dapat memilih untuk mengurutkan daftar secara naik atau menurun, tergantung pada kebutuhan kita. Selain itu, *Insertion Sort* juga dapat digunakan untuk mengurutkan daftar yang mengandung elemen-elemen dengan nilai yang sama.

Gambar 3.5 menunjukkan contoh penerapan *Insertion Sort* untuk mengurutkan suatu kumpulan data dalam array. Perulangan pertama memeriksa bilangan di posisi ke-2. Nilai 22 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri dan langsung ditemukan bahwa bilangan 16 di sebelahnya sudah lebih kecil dari 22.Tidak terjadi penyisipan.

Perulangan kedua berdasarkan Gambar 3.5 memeriksa bilangan di posisi ke-3. Nilai 25 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri dan langsung ditemukan bahwa bilangan 22 disebelahnya sudah lebih kecil dari 25. Tidak terjadi penyisipan.

Perulangan ketiga berdasarkan Gambar 3.5 memeriksa bilangan di posisi ke-4. Nilai 3 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri

Kondisi Awal

1	2	3	4	5	6	7
16	22	25	3	9	14	23

INSERTION SORT

1

1	2	3	4	5	6	7
16	22	25	3	9	14	23

2

1	2	3	4	5	6	7
16	22	25	3	9	14	23

3

1	2	3	4	5	6	7
3	16	22	25	9	14	23

4

1	2	3	4	5	6	7
3	9	16	22	25	14	23

5

1	2	3	4	5	6	7
3	9	14	16	22	25	23

6

1	2	3	4	5	6	7
3	9	14	16	22	23	25

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Hasil Akhir

Gambar 3.5: Ilustrasi algoritme Insertion Sort

dan langsung ditemukan bahwa bilangan 25 di sebelahnya lebih besar. Program terus mencari ke kiri sampai ditemukan bilangan yang lebih kecil daripada 3. Tidak ditemukan. Maka isi array posisi ke-1 sampai 3 semuanya bergeser ke kanan. Nilai 3 dimasukkan ke posisi indeks ke-1 dari array.

Perulangan keempat berdasarkan Gambar 3.5 memeriksa bilangan di posisi ke-5. Nilai 9 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri dan langsung ditemukan bahwa bilangan 25 di sebelahnya lebih besar. Program terus mencari ke kiri sampai ditemukan bilangan yang lebih kecil dari pada 9. Ditemukan bahwa isi A[1] lebih kecil dari pada 9. Isi array ke-2 sampai ke-4 bergeser ke kanan agar nilai 9 dapat disisipkan ke posisi indeks ke-2.

Perulangan kelima berdasarkan Gambar 3.5 memeriksa bilangan di posisi ke-6. Nilai 14 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri dan langsung ditemukan bahwa bilangan 25 disebelahnya lebih besar. Program terus mencari ke kiri sampai ditemukan bilangan yang lebih kecil dari 14. Ditemukan bahwa isi A[2] lebih kecil daripada 9. Isi array ke-3 sampai 5 bergeser ke kanan agar nilai 14 dapat disisipkan ke posisi indeks ke-3.

Perulangan keenam berdasarkan Gambar 3.5 memeriksa bilangan di posisi ke-7. Nilai 23 disimpan dalam variabel *iDataSisip*. Kemudian program mencari ke kiri dan langsung ditemukan bahwa bilangan 25 di sebelahnya lebih besar. Program terus mencari ke kiri sampai ditemukan bilangan yang lebih kecil dari 23. Ditemukan bahwa isi A[5] lebih kecil daripada 23. Isi array ke-6 bergeser ke kanan agar nilai 23 dapat disisipkan ke posisi indeks ke-6.

3.3 Bubble Sort

Bubble Sort adalah salah satu algoritme pengurutan sederhana yang bekerja dengan cara membandingkan dan menukar elemen-elemen *adjacent* satu per satu hingga seluruh daftar terurut. Nama *"Bubble"* muncul karena elemen-elemen yang lebih kecil "naik" ke atas daftar, mirip gelembung yang naik ke permukaan air. Nama algoritma *Bubble Sort* diambil dari ide dasar algoritma ini yang bekerja berdasarkan prinsip gelembung. Teknik Bubble sort menganalogikan setiap angka sebagai sebuah gelembung. Angka yang lebih kecil akan naik ke atas sama seperti gelembung yang selalu akan mengambang di atas permukaan air.

Dalam setiap perulangan i, algoritma akan “menggelembungkan” bilangan terkecil ke i hingga ke posisi ke-i dalam array. Penggelembungan dimulai dari ele-

```
for i=1 to (N-1) do
  for j=N downto i-1 do
    if A[j-1] > A[j] then
      swap(A[j-1],A[j])
    end if
  end for
end for
```

Gambar 3.6: Algoritme Bubble Sort

men terakhir array. Setiap pasang elemen yang berdekatan akan dibandingkan. Jika elemen yang dibelakang lebih kecil, maka dilakukan penukaran. Dengan cara ini, bilangan terkecil pasti akan "naik terbawa" hingga ke posisi sebenarnya.

Penting untuk dicatat bahwa *Bubble Sort* memiliki kompleksitas waktu yang buruk, yaitu $O(n^2)$ dalam kasus terburuk, di mana n adalah jumlah elemen dalam daftar. Oleh karena itu, untuk daftar yang besar, algoritme pengurutan lain seperti *Merge Sort* atau *Quick Sort* seringkali lebih disukai karena memiliki kompleksitas waktu yang lebih baik.

Gambar 3.7 menunjukkan contoh implementasi dari algoritme *Bublle Sort*, sedangkan Gambar 3.8 s.d 3.15 menunjukkan detil langkah berdasarkan ilustrasi pada Gambar 3.7.

3.4 Tantangan

Buatlah fungsi dalam bahasa pemrograman Python untuk mengimplementasikan konsep struktur data pada algoritme-algoritme sorting yang sudah dijelaskan dalam modul!

Kondisi awal

1	2	3	4	5	6	7
16	22	25	3	9	14	23

1	2	3	4	5	6	7	
16	22	25	3	9	14	23	1a
16	22	25	3	9	14	23	1b
16	22	25	3	9	14	23	1c
16	22	3	25	9	14	23	1d
16	3	22	25	9	14	23	1e
3	16	22	25	9	14	23	1f
3	16	22	25	9	14	23	2a
3	16	22	25	9	14	23	2b
3	16	22	9	25	14	23	2c
3	16	9	22	25	14	23	2d
3	9	16	22	25	14	23	2e
3	9	16	22	25	14	23	3a
3	9	16	22	14	25	23	3b
3	9	16	14	22	25	23	3c
3	9	14	16	22	25	23	3d
3	9	14	16	22	23	25	4a
3	9	14	16	22	23	25	4b
3	9	14	16	22	23	25	4c
3	9	14	16	22	23	25	5a
3	9	14	16	22	23	25	5b
3	9	14	16	22	23	25	5c

BUBBLE SORT

1	2	3	4	5	6	7	
3	9	14	16	22	23	25	Hasil akhir

Gambar 3.7: Ilustrasi Bubble Sort

Kondisi awal *array A*

1	2	3	4	5	6	7
16	22	25	3	9	14	23

Step 1a. Bandingkan 14 dengan 23

1	2	3	4	5	6	7
16	22	25	3	9	14	23

Step 1b. Bandingkan 9 dengan 14

1	2	3	4	5	6	7
16	22	25	3	9	14	23

Step 1c. Bandingkan 3 dengan 9

1	2	3	4	5	6	7
16	22	25	3	9	14	23

Gambar 3.8: step 1a s.d step 1c

Step 1d. Bandingkan 25 dengan 3. Tukar posisinya.

1	2	3	4	5	6	7
16	22	3	25	9	14	23

Step 1e. Bandingkan 22 dengan 3. Tukar posisinya.

1	2	3	4	5	6	7
16	3	22	25	9	14	23

Step 1f. Bandingkan 16 dengan 3. Tukar posisinya. Nilai 3 telah "menggelembung" Sampai posisi pertama

1	2	3	4	5	6	7
3	16	22	25	9	14	23

Gambar 3.9: step 1d s.d step 1f

Step 2a. Bandingkan 14 dengan 23.

1	2	3	4	5	6	7
3	16	22	25	9	14	23

Step 2b. Bandingkan 9 dengan 14.

1	2	3	4	5	6	7
3	16	22	25	9	14	23

Step 2c. Bandingkan 25 dengan 9. Tukar posisinya.

1	2	3	4	5	6	7
3	16	22	9	25	14	23

Gambar 3.10: step 2a s.d step 2c

Step 2d. Bandingkan 22 dengan 9. Tukar posisinya.

1	2	3	4	5	6	7
3	16	9	22	25	14	23

Step 2e. Bandingkan 16 dengan 9. Tukar posisinya. Nilai 9 kini telah berada Pada posisi yang benar.

1	2	3	4	5	6	7
3	9	16	22	25	14	23

Gambar 3.11: step 2d s.d step 2e

Step 3a. Bandingkan 14 dengan 23.

1	2	3	4	5	6	7
3	9	16	22	25	14	23

Step 3b. Bandingkan 25 dengan 14. Tukar posisinya.

1	2	3	4	5	6	7
3	9	16	22	14	25	23

Gambar 3.12: step 3a s.d step 3b

Step 3c. Bandingkan 22 dengan 14. Tukar posisinya.

1	2	3	4	5	6	7
3	9	16	14	22	25	23

Step 3d. Bandingkan 16 dengan 14. Tukar posisinya. Nilai 14 kini telah di posisi yang benar.

1	2	3	4	5	6	7
3	9	14	16	22	25	23

Gambar 3.13: step 3c s.d step 3d

Step 4a. Bandingkan 25 dengan 23. Tukar posisinya.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Step 4b. Bandingkan 22 dengan 23.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Step 4c. Bandingkan 16 dengan 22. Nilai 16 kini telah di posisi yang benar.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Gambar 3.14: step 4a s.d step 4c

Step 5a. Bandingkan 23 dengan 25.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Step 5b. Bandingkan 22 dengan 23. Nilai 22 kini telah di posisi yang benar.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Step 6. Bandingkan 23 dengan 25. Nilai 23 kini telah di posisi yang benar. Nilai 25 yang Berada di posisi terakhir secara otomatis dapat disimpulkan sudah berada di posisi Yang benar pula.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Hasil akhir.

1	2	3	4	5	6	7
3	9	14	16	22	23	25

Gambar 3.15: step 5a s.d hasil akhir